

Relatório - Lista 3

Equipe: Angel Jhesua Machado Rodriguez e João Guilherme de Souza Gomes

Disciplina: Estrutura de Dados

Professor: Matheus Telles Werner

Mai de 2026

Implementação dos algoritmos de ordenação

Para ambos algoritmos usamos a função `comesFirst`, que recebe como primeiro argumento um player a , como segundo argumento um player b , então compara o jogador a com o b para determinar a precedência na fila tendo em conta o score e o timestamp em casos de mesmo score. Retorna true se a deve preceder b , false caso contrário.

Insertion Sort

Implementamos ele em apenas uma função `sortByScoreInsertion`. Inicialmente consideramos a fila desordenada de players em espera com n player então selecionamos o primeiro elemento dela como nossa fila ordenada e o segundo elemento dela como nosso `current` então comparamos nosso `current` com cada elemento da nossa fila ordenada, do maior pro menor, usando a função `comesFirst`, achamos a posição quando o `current` não possa vir primeiro do que algum elemento da fila ordenada. então nossa fila ordenada agora tem 2 elementos e a fila desordenada tem $n-2$ elementos. Dessa forma inserimos cada elemento do sub arranjo desordenado no sub arranjo ordenado.

Merge Sort

Implementamos ele com 3 funções: `merge`, `mergeSort`, `sortByScoreMerge`.

`merge` recebe os seguintes parâmetros:

`Player* arr, Player* aux, int start, int mid, int end`

`arr` é o array de players no qual existem dois sub arranjos ordenados, com índices `start`, `mid` e `mid + 1`, `end` queremos então juntar esses dois sub arranjos ordenados em um único arranjo ordenado. `aux` é um array onde colocamos os elementos ordenados, comparando sempre o topo dos dois sub arranjos com a função `comesFirst`, para depois copiar eles para `arr`

`mergeSort` recebe os seguintes parâmetros:

`mergeSort(Player* arr, Player* aux, int start, int end)`

`arr` é o array que possui o sub arranjo com índices `start`, `end` que queremos ordenar, `aux` é o array que usaremos para cada merge, como para o maior merge precisamos um `aux` com tamanho `end - 1` para os índices de `aux`. baterem com os índices de `arr` então `aux` é um vetor de tamanho `size` onde `size` é o numero de players na fila de espera.

Para conseguir a ordenação usamos uma recursão, com os seguintes passos:

1. Criamos o caso base, nosso algoritmo para se o sub arranjo é apenas um elemento (logo já esta ordenado) ou se foram passados índices de sub arranjo impossíveis (inicio maior que o final)
2. Dividimos o problema pela metade, em dois sub arranjos, esquerdo e direito, resolvemos então de maneira recursiva esses sub problemas ate chegar no caso base
3. Finalmente combinamos a solução dos subproblemas usando `merge`

O `sortByScoreMerge` foi apenas uma função para criar o vetor `aux` de maneira dinâmica.

Como a ordenação foi aproveitada para formar grupos

Usamos uma abordagem de janela deslizante. Selecionamos os primeiros n players, se não for possível criar um grupo, pela fila estar ordenada então podemos olhar para os primeiros n players pulando o primeiro, e assim sucessivamente, ate achar um grupo ou ate que não seja possível selecionar n players pulando os k primeiros.

Custo computacional das principais operações do sistema

- Inserção (**insert**): $O(1)$

A operação é de tempo constante, pois o novo jogador é sempre adicionado ao final do array utilizando o índice 'size', sem necessidade de deslocar os elementos existentes.

- Remoção (**removePlayer**): $O(n)$

O algoritmo realiza uma busca linear pelo **id** do jogador. No pior caso (remover o primeiro elemento), é necessário deslocar todos os $n - 1$ elementos subsequentes uma posição para a esquerda para preencher o espaço.

- Ordenação com Insertion Sort (**sortByScoreInsertion**): $O(n^2)$

Composto por dois laços aninhados. No pior caso (quando os jogadores estão na ordem inversa), cada novo elemento avaliado precisa ser comparado e deslocado por todos os elementos já ordenados no início do array.

- Ordenação com Merge Sort (**sortByScoreMerge**): $O(n \log n)$

O array é dividido recursivamente pela metade ($O(\log n)$) e as sublistas são combinadas em tempo linear ($O(n)$). Vale ressaltar que este método exige um custo de espaço (memória) adicional de $O(n)$ para a alocação do array auxiliar **aux**.

- Formação de Grupo (**formGroup**): $O(n)$

A função utiliza uma abordagem de janela deslizante que percorre o array uma vez. Caso um grupo válido seja encontrado, a função **removeSortedPlayers** é chamada, o que desloca os elementos restantes também em tempo linear. Como essas operações de deslocamento e iteração não são multiplicativas, o custo no pior caso permanece linear.

- Recuperação dos Jogadores (**getWaitingPlayers**): $O(n)$

Para retornar a lista, o método aloca dinamicamente um novo array e itera por todos os **n** jogadores atuais do sistema para copiá-los um a um, caracterizando um custo linear.